

부산공유대학 미래모빌리티전공

파이썬 프로그래밍

소속 : 동서대학교 컴퓨터공학과



1 File Handling

Python File Open

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting files.

File Handling

The key function for working with files in Python is the open() function.

The open() function takes two parameters; *filename*, and *mode*.

There are four different methods (modes) for opening a file:

- "r" – Read – Default value. Opens a file for reading, error if the file does not exist
- "a" – Append – Opens a file for appending, creates the file if it does not exist
- "w" – Write – Opens a file for writing, creates the file if it does not exist
- "x" – Create – Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode

- "t" – Text – Default value. Text mode
- "b" – Binary – Binary mode (e.g. images)

1 File Handling

Syntax

To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

The code above is the same as:

```
f = open("demofile.txt", "rt")
```

Because "r" for read, and "t" for text are the default values, you do not need to specify them.

Note: Make sure the file exists, or else you will get an error.

1 File Handling

Python File Open

Assume we have the following file, located in the same folder as Python:

demofile.txt

Hello! Welcome to demofile.txt

This file is for testing purposes. To open the file, use the built-in open() function.

The open() function returns a file object, which has a read() method for reading the content of the file:

Example

```
f = open("demofile.txt")  
print(f.read())
```

If the file is located in a different location, you will have to specify the file path, like this:

Example Open a file on a different location:

```
f = open("D:\\myfiles\\welcome.txt")  
print(f.read())
```

1 File Handling

Using the with statement

You can also use the with statement when opening a file:

Example Using the with keyword:

```
with open("demofile.txt") as f:  
    print(f.read())
```

Then you do not have to worry about closing your files, the with statement takes care of that.

1 File Handling

Close Files

It is a good practice to always close the file when you are done with it.

If you are not using the with statement, you must write a close statement in order to close the file:

Example Close the file when you are finished with it:

```
f = open("demofile.txt")  
print(f.readline())  
f.close()
```

Note: You should always close your files. In some cases, due to buffering, changes made to a file may not show until you close the file.

1 File Handling

Read Only Parts of the File

By default the read() method returns the whole text, but you can also specify how many characters you want to return:

Example Return the 5 first characters of the file:

```
with open("demofile.txt") as f:  
    print(f.read(5))
```

1 File Handling

Read Lines

You can return one line by using the `readline()` method:

Example Read one line of the file:

```
with open("demofile.txt") as f:  
    print(f.readline())
```

By calling `readline()` two times, you can read the two first lines:

Example Read two lines of the file:

```
with open("demofile.txt") as f:  
    print(f.readline())  
    print(f.readline())
```

By looping through the lines of the file, you can read the whole file, line by line:

Example Loop through the file line by line:

```
with open("demofile.txt") as f:  
    for x in f:  
        print(x)
```


1 File Handling

Python File Write

Write to an Existing File

To write to an existing file, you must add a parameter to the open() function:

"a" – Append – will append to the end of the file

"w" – Write – will overwrite any existing content

Example Open the file "demofile.txt" and append content to the file:

```
with open("demofile.txt", "a") as f:
```

```
    f.write("Now the file has more content!")
```

#open and read the file after the appending:

```
with open("demofile.txt") as f:
```

```
    print(f.read())
```

1 File Handling

Overwrite Existing Content

To overwrite the existing content to the file, use the w parameter:

Example Open the file "demofile.txt" and overwrite the content:

```
with open("demofile.txt", "w") as f:
    f.write("Woops! I have deleted the content!")
#open and read the file after the overwriting:
with open("demofile.txt") as f:
    print(f.read())
```

Note: the "w" method will overwrite the entire file.

1 File Handling

Create a New File

To create a new file in Python, use the open() method, with one of the following parameters:

"x" – Create – will create a file, returns an error if the file exists

"a" – Append – will create a file if the specified file does not exists

"w" – Write – will create a file if the specified file does not exists

Example Create a new file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created.

Note: If the file already exist, an error will be raised.

1 File Handling

Delete a File

To delete a file, you must import the OS module, and run its `os.remove()` function:

Example Remove the file "demofile.txt":

```
import os  
os.remove("demofile.txt")
```

1 File Handling

Check if File exist:

To avoid getting an error, you might want to check if the file exists before you try to delete it:

Example Check if file exists, *then* delete it:

```
import os
if os.path.exists("demofile.txt"):
    os.remove("demofile.txt")
else:
    print("The file does not exist")
```

1 File Handling

Overwrite Existing Content

To overwrite the existing content to the file, use the w parameter:

Example Open the file "demofile.txt" and overwrite the content:

```
with open("demofile.txt", "w") as f:  
    f.write("Woops! I have deleted the content!")  
#open and read the file after the overwriting:  
with open("demofile.txt") as f:  
    print(f.read())
```

Note: the "w" method will overwrite the entire file.

1 File Handling

Delete Folder

To delete an entire folder, use the `os.rmdir()` method:

Example Remove the folder "myfolder":

```
import os  
os.rmdir("myfolder")
```

Note: You can only remove *empty* folders.